Assignment No - 10

Title:

Analyze the HTTP security headers of an authorized website, identify missing or weak security protections, assess the possible risks, and suggest suitable security recommendations.

1. Implementation

To perform the problem statement, the HTTP response headers of an authorized website were inspected using command-line tools. The analysis focused on identifying security-related HTTP response headers and checking whether important browser security protections were present, missing, or weak.

- The authorized website URL was selected for testing.
- HTTP response headers were collected using curl.
- Security-related headers were identified from the response.
- Missing or weak protections were recorded for risk assessment.
- Suitable security recommendations were prepared based on the observed configuration.

2. Important Commands

2.1 Display HTTP Response Headers

```
curl -I https://example.com
```

Displays the HTTP response headers returned by the website.

2.2 Follow Redirects

```
curl -I -L https://example.com
```

Follows HTTP redirects and displays headers from the request chain.

2.3 Check a Security Header

```
curl -I https://example.com | findstr /I "Content-Security-Policy"
```

Searches the response for a specific security header.

2.4 Check Multiple Headers

```
curl -I https://example.com | findstr /I "Strict-Transport-Security Content-Security-Policy X-Frame-Options X-Content-Type-Options Referrer-Policy Permissions-Policy"
```

Check whether important security headers are present.

3. Theory / Background

HTTP security headers are response headers sent by a web server to a browser. They provide security-related instructions that influence how the browser loads, executes, embeds, caches, and references web content. These controls mainly strengthen browser-side defenses and can reduce the impact of several common web attacks.

Security-header analysis is a configuration review. It should be performed only on authorized systems and should be combined with broader application-security testing. A missing header is a finding that needs context; it is not, by itself, proof that the application is exploitable.

3.1 Purpose of Security Headers

The main purposes of HTTP security headers are to reduce unsafe browser behavior, limit the sources from which content may be loaded, protect pages from unwanted framing, enforce secure transport, reduce information leakage, and restrict access to browser features that an application does not require.

3.2 Key HTTP Security Headers

Strict-Transport-Security (HSTS)

HSTS tells a compatible browser to use HTTPS for future connections to the host for a specified period. It helps reduce downgrade and insecure-transport scenarios after the policy has been received. It is normally configured with max-age and may include includeSubDomains.

Content-Security-Policy (CSP)

CSP defines which sources the browser may use for scripts, styles, images, frames, and other resources. A carefully designed policy can reduce the impact of cross-site scripting and related injection attacks. The policy must match the application's real resource requirements.

X-Frame-Options

X-Frame-Options controls whether a page can be rendered inside a frame. DENY blocks framing and SAMEORIGIN allows framing by the same origin. It is commonly used against clickjacking.

X-Content-Type-Options

The value nosniff tells compatible browsers not to infer a different MIME type from the server-declared content type. It reduces certain MIME-type sniffing risks.

Referrer-Policy

Referrer-Policy controls how much URL information is sent in the Referer header during navigation. A suitable policy can reduce accidental disclosure of sensitive path or query information.

Permissions-Policy

Permissions-Policy can restrict browser capabilities such as camera, microphone, geolocation, and other powerful features. Restricting unused features reduces unnecessary browser attack surface.

Cookie Security Attributes

Secure limits cookie transmission to HTTPS, HttpOnly prevents normal client-side scripts from reading the cookie, and SameSite controls cross-site cookie sending behavior.

Cache-Control

Cache-Control determines how a response may be cached. Sensitive, authenticated responses may require restrictive caching so confidential information is not stored by shared or intermediary caches.

3.3 Common Risks from Missing or Weak Headers

- XSS impact: Missing or weak CSP can remove a useful browser-side layer of defense when script injection is possible.
- Clickjacking: Missing framing restrictions can allow a sensitive page to be embedded by another site.
- Transport security: Missing HSTS reduces browser-enforced HTTPS persistence after the policy should have been established.
- Information disclosure: An unsuitable Referrer-Policy may disclose more URL information than required.
- Unnecessary browser features: A permissive Permissions-Policy can leave capabilities enabled that the application does not use.
- Content sniffing: Missing X-Content-Type-Options can permit MIME-type guessing in browsers that support sniffing behavior.
- Sensitive caching: Weak cache directives can result in private responses being stored where they should not be.

3.4 Header Assessment Methodology

- Define the authorized target and testing scope.
- Collect response headers using curl or browser developer tools.
- Record the exact name and value of each relevant header.
- Compare the configuration with application security and functional requirements.
- Classify findings as present, missing, or weakly configured.
- Assess possible impact without assuming that a missing header proves exploitability.
- Recommend a suitable configuration and retest after remediation.

3.5 Limitations of Header-Only Assessment

HTTP security-header analysis is not a complete penetration test. A website can contain strong header protections and still have vulnerabilities in authentication, authorization, input validation, session management, APIs, or business logic. Conversely, a missing header may be acceptable in a specific

architecture. Therefore, header findings should be correlated with the application's wider security controls.

4. Performance / Practical Execution

The following terminal outputs are sample outputs prepared for practical documentation. The target should be replaced with an authorized website when performing the actual practical.

4.1 HTTP Header Collection

```
Command Prompt
C:\Users\shrad>curl -I https://example.com

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
Server: nginx
X-Content-Type-Options: nosniff

C:\Users\shrad>curl -I https://example.com | findstr /I Security
Strict-Transport-Security: MISSING
Content-Security-Policy: MISSING
X-Frame-Options: MISSING
```

Figure 1: Sample terminal output showing HTTP response headers.

4.2 Security Header Verification

```
Command Prompt
C:\Users\shrad>curl -I https://example.com

HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
Referrer-Policy: MISSING
Permissions-Policy: MISSING
Strict-Transport-Security: MISSING
Content-Security-Policy: MISSING
X-Frame-Options: MISSING

Security Header Assessment: Review Required
```

Figure 2: Sample terminal output showing the security-header review.

Observed Details

Security Header	Observed Status	Security Purpose
Strict-Transport-Security	Missing	Forces HTTPS connections.

Content-Security-Policy	Missing	Restricts content sources and helps reduce XSS impact.
X-Frame-Options	Missing	Helps prevent clickjacking.
X-Content-Type-Options	Present: nosniff	Reduces MIME-type sniffing.
Referrer-Policy	Missing	Controls referrer information.
Permissions-Policy	Missing	Restricts browser capabilities.

5. Analysis

The sample response contains several security-related observations. X-Content-Type-Options is present with the value nosniff, while several commonly used security headers are missing.

The absence of Strict-Transport-Security means the browser is not instructed through this header to enforce HTTPS for future connections.

The absence of Content-Security-Policy removes an important browser-side control for restricting script and resource sources and can increase the potential impact of an XSS vulnerability.

The missing X-Frame-Options header removes this header-based framing protection and may increase exposure to clickjacking depending on other controls.

The missing Referrer-Policy and Permissions-Policy headers indicate that additional browser privacy and feature-control protections have not been explicitly configured.

Header absence alone does not prove that a website is vulnerable. Risk depends on the application's architecture and other security controls.

6. Security Recommendations

- Enable Strict-Transport-Security after confirming that HTTPS is correctly supported.
- Deploy a carefully designed Content-Security-Policy and test it before enforcement.
- Use X-Frame-Options or CSP frame-ancestors where appropriate.
- Keep X-Content-Type-Options: nosniff enabled.
- Configure an appropriate Referrer-Policy.
- Configure Permissions-Policy to restrict unused browser features.
- Review Secure, HttpOnly, and SameSite attributes on session cookies.
- Repeat header testing after configuration changes.

7. Result

- HTTP response headers were collected using curl.
- Security-related headers were identified and reviewed.
- Missing and present protections were documented.
- Potential risks were assessed.
- Suitable security recommendations were prepared.

8. Conclusion

The practical demonstrated how HTTP response headers can be analyzed to identify security protections configured by a web application. Headers such as Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy provide different layers of browser-side security.

Missing or weak headers should be evaluated together with the application's other security controls. Implementing suitable security headers, validating their configuration, and periodically reassessing them can improve the security posture of an authorized web application.